



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF INFORMATION TECHNOLOGY

Development of a software control platform for a remote-controlled car

Supervisor:

Sándor Laki

Assistant Lecturer

Author:

Franciska Jázmin Czobor

Computer Science BSc

Budapest, 2026

Abstract

This thesis presents an expandable platform for a remote-controlled car centered around a single-board computer. The project features a control platform and a simple line-follower application. It utilises WebRTC to provide a low-latency live video feed to the operator, and WebSocket for control signal transmission.

The software architecture follows a modular design, featuring a relay server that allows selective sensor data subscription. To demonstrate the platform's utility, an autonomous line-follower application was developed using OpenCV.

The primary objective of this work was to establish an extensible environment that facilitates future research projects, educational demonstration, and further development of autonomous features.

Contents

Abstract	1
1 Introduction	3
1.1 Project objectives	3
1.2 Project overview	4
1.2.1 Edge server (Car)	4
1.2.2 Relay server	6
1.2.3 Client	7
1.3 Thesis outline	8
2 User documentation	9
2.1 Project structure	9
2.2 Installation	10
2.3 Running the components	10
2.4 Application usage	11
2.4.1 Manual control application	11
2.4.2 Line-follower application	13
3 Developer documentation	15
3.1 Project structure	15
3.1.1 Configuration	16
3.2 Network architecture	17
3.2.1 Media pipeline	18
3.2.2 Control pipeline	21
3.2.3 Telemetry pipeline	23
3.3 Implementation	25
3.3.1 Edge server	25
3.3.2 Relay server	33

3.3.3	Manual control application	35
3.3.4	Line-follower application	37
3.4	Testing	42
3.4.1	Unit tests	42
3.4.2	Manual tests	44
4	Conclusion	46
4.1	Future plans	47
	Acknowledgements	48
	Bibliography	48
	List of Figures	50

Chapter 1

Introduction

A remote-controlled (RC) car can be a highly effective tool for evaluating and demonstrating various engineering concepts, ranging from simple programs to autonomous navigation or Internet of Things (IoT) integration.

This thesis demonstrates a software application designed to control an RC car remotely over web connections. The primary objective of this work was to establish an extensible system, predisposed for future development.

1.1 Project objectives

To achieve a scalable and responsive platform, this project focused on the following technical objectives:

- **Expandable architecture:** To develop a relay server capable of seamlessly routing data between the physical vehicle and any client, regardless of the implementation of the client.
- **Low-latency communication:** To design and implement a robust network pipeline utilising WebSockets for control transmission and WebRTC for high-speed video transmission.
- **Hardware abstraction:** To create a modular hardware abstraction layer on the edge node, effectively translating digital control commands into physical electrical signals.
- **Autonomous line-following:** To demonstrate the platform's capabilities by implementing an autonomous line-follower client.

1.2 Project overview

The proposed system is structured as a distributed architecture consisting of three primary nodes:

1.2.1 Edge server (Car)

The physical component of our system is a remote-controlled vehicle that functions as the network's edge node. Rather than relying on traditional radio-frequency transmitters, the vehicle has been mounted with a microcomputer, making the car highly programmable and internet-enabled.

For the purposes of this thesis, the term '**car**' (or '**edge server**') refers to the collection of the following components:

- RC car (150603 Phantom MT [1])
- Microcomputer (Raspberry Pi 3 B+ [2])
- Camera (Genius USB camera)

Although these components form the foundational hardware stack, the edge node is designed with a modular architecture. This extensible design ensures that future sensors and actuators can be seamlessly integrated into the system with minimal changes to configurations.

While the microcomputer manages high-level network logic and image processing, it lacks the ability (specialised hardware) to directly control the motors or the servo. To solve this, the microcomputer communicates with a secondary controller - a PCA9685 driver - which acts as a bridge between digital software commands and motor signals. This driver then sends precise Pulse-Width Modulation signals to two critical endpoints: the Electronic Speed Controller (ESC), which regulates power from the battery to the drive motor, and the steering servo, which executes the directional changes.

This internal communication and the succeeding response are governed by two fundamental protocols:

Inter-Integrated Circuit Bus (I2C)

I2C is a synchronous, multi-target serial communication bus protocol. It is used for connecting peripheral integrated circuits to processors and microcontrollers.

Within this system's architecture, the Raspberry Pi functions as the I2C controller. It utilises the `smbus2` library to transmit localised steering and throttle values over the hardware bus to the PCA9685 PWM driver.

Pulse-Width Modulation (PWM)

Pulse-Width Modulation (PWM) is a method for simulating analog signals with digital ones. Digital signals have strictly two states (**HIGH** and **LOW**), and by rapidly alternating between the two, we can achieve analog-like results.

While in standard electronics, the primary characteristic of a PWM signal is its duty cycle (the percentage of time the signal remains in the **HIGH** state), RC servos and electronic speed controllers (ESC) rely strictly on the duration of time the signal remains in the **HIGH** state, measured in microseconds.

In the context of this project, these values are interpreted as the magnitude of the speed and steering of the vehicle.

1.2.2 Relay server

While the car and the client represent the endpoints of the system, the relay server acts as the central coordinator of the communication pipeline. It utilises two primary communication protocols: WebSocket for control data, and WebRTC for video transmission.

The WebSocket Protocol

The WebSocket protocol provides a persistent, full-duplex communication channel over a single TCP connection. It allows for continuous, two-way data transfer with minimal latency.

Web Real-Time Communication (WebRTC)

WebRTC is an open-source framework that enables peer-to-peer communication of audio, video, and other data across the web. Rather than using centralised streaming servers, WebRTC establishes direct connections between clients.

- **Interactive Connectivity Establishment (ICE):** To find a routable network path, WebRTC uses the ICE framework. ICE systematically gathers various network addresses, known as ICE Candidates.
- **ICE Candidates:** An ICE candidate is a string of text representing a potential network path.
- **Trickle ICE:** With Trickle ICE, the peer sends its initial WebRTC offer immediately and continues gathering network paths in the background. As soon as a new ICE candidate is discovered, it is "trickled" across the signaling channel. The receiving peer tests these candidates asynchronously, establishing the media stream the moment a valid path is verified, thereby minimising connection delays.
- **STUN server:** A STUN server simply echoes back the public IP address and port from which it received a query. This allows a device behind a router to discover its own public-facing address.

1.2.3 Client

In this distributed architecture, the client refers to the software endpoint responsible for issuing control commands and interacting with the edge node via the relay server. While client applications can vary significantly in their implementation, they share a unified communication paradigm. Any client in this system must connect to the relay server, negotiate the WebRTC media stream, and use the WebSocket pipeline to exchange data in a `json` format.

Manual control application

The primary function of the manual control interface is to translate human inputs into structured, machine-readable data packets.

Based on an event-driven JavaScript architecture, the application captures keyboard inputs, and sends the control packets over the WebSocket connection to the relay server. Simultaneously, the application renders the incoming WebRTC video feed and displays information about the vehicle's state. This creates a tight feedback loop, allowing the operator to adjust their inputs based on visual and telemetric data.

Line-follower application

To demonstrate the versatility of the network architecture, the line-follower application functions as an autonomous, programmatic client that can follow a path (line). Rather than relying on human input, this standalone application processes the vehicle's WebRTC video stream using OpenCV, dynamically calculates the necessary control commands, and transmits them back through the WebSocket pipeline.

The line-follower integrates into the system using the exact same network pathways as the manual client. It also displays the processed video feed, visualising how the algorithm works.

1.3 Thesis outline

The remainder of this thesis is structured to guide the reader through the practical usage, architecture, and technical implementation of the system.

- Chapter 2, **User documentation**, outlines the practical steps required to install the software, initialise the components, and use both client applications.
- Chapter 3, **Developer documentation**, explores the system's internal network architecture, breaks down the implementation and introduces the testing methodologies.
- Chapter 4, **Conclusion**, summarises the project's technical achievements and reflects on the overall results of the work.

Chapter 2

User documentation

This section provides instructions for setting up and operating the software.

2.1 Project structure

The directory is organised into distinct modules for the client interfaces, the relay server, and the car. A simplified overview of the directory is provided below:

```
picar/
├── pyproject.toml
├── picar_core/
│   ├── control_app/
│   │   ├── client.html
│   │   ├── config.js
│   │   └── ... (supporting scripts)
│   ├── linefollower_app/
│   │   ├── linefollower.py
│   │   └── line_config.json
│   ├── pc/
│   │   ├── pc_config.json
│   │   └── ... (relay server scripts)
│   ├── pi/
│   │   ├── pi_config.json
│   │   └── ... (car scripts)
└── ... (tests and environment configurations)
```

2.2 Installation

The following instructions describe the deployment of the system across its three primary network nodes, as defined in Section 1.2. While functionally distinct components may be hosted on a single machine for testing, this guide details a distributed configuration involving the following nodes:

- **Relay Node:** The central server.
- **Edge Node:** The Raspberry Pi (the car).
- **Client Node:** The machine used for manual operation or autonomous line-following.

Before proceeding, ensure that Python 3.10 or a later version is installed on all participating nodes.

Install the project and the dependencies listed in `pyproject.toml` on all the machines by running the following command from the project root directory:

```
# cd path/to/picar
pip install .
```

2.3 Running the components

Once the installations are completed, start the server components using the provided entry points defined in `pyproject.toml` by executing the subsequent instructions.

- On the edge node:

```
pi-server
```

- On the relay node:

```
pc-server
```

2.4 Application usage

Following the initialisation of the core components, the system supports two main applications: manual navigation via the control application or autonomous movement using a line-follower algorithm.

2.4.1 Manual control application

Access the manual control interface by opening `client.html` in a web browser on the client machine.

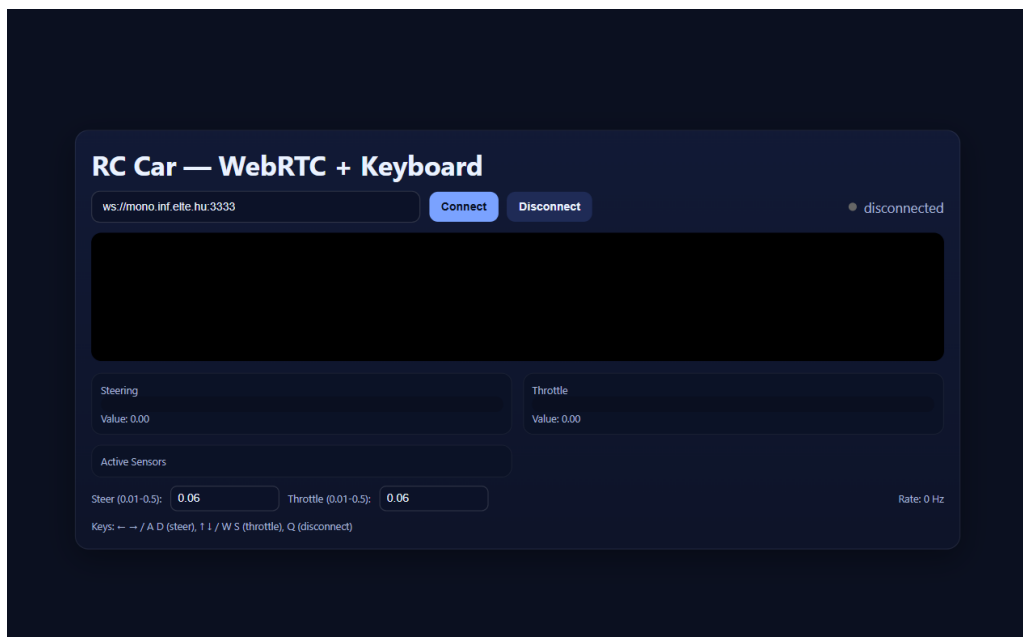


Figure 2.1: The client before connecting to the server

The server address is preconfigured (as seen in Figure 2.1). This preset can be changed in `picar_core/config.js`. By pressing the **Connect** button, connection to the relay server is initiated.

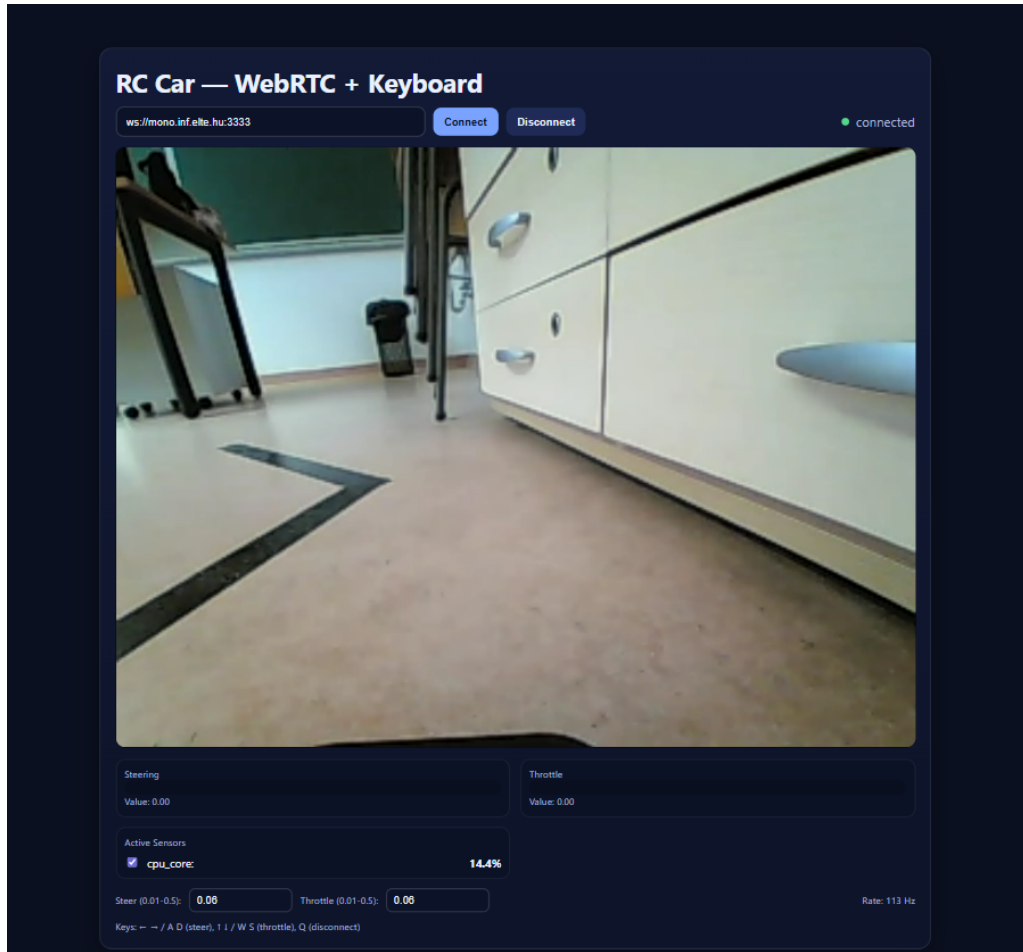


Figure 2.2: The client after connecting to the server

The connection status can be seen in the upper right corner, indicated by a green (connected) or red (disconnected) disk. After successful connection, the live camera view is visible (as demonstrated in Figure 2.2).

Keyboard controls

The vehicle is operated via keyboard inputs, the specific mappings are detailed in Table 2.1.

Key	Action
A / ←	Steer left
D / →	Steer right
W / ↑	Throttle forward
S / ↓	Throttle backward
Q	Disconnect client

Table 2.1: Keyboard control mapping

Sensor management

Situated below the camera feed is the sensor management panel, where sensors are selectable via checkboxes. By clicking a checkbox, a request is sent to the car server to periodically send data measured by the sensor. The results are indicated to the right of the chosen checkbox. Multiple sensors can be selected at the same time.

Other parameters

The following parameters can also be set through the client:

Step (steer) Defines the steering sensitivity. A larger step results in faster wheel movement and sharper turns.

Step (throttle) Defines the acceleration rate. Higher values allow the vehicle to reach target speeds more rapidly.

2.4.2 Line-follower application

This section serves as a functional guide for the autonomous line-follower module. It describes the necessary environmental conditions for reliable path detection and provides instructions for running the application.

Environmental requirements

The system requires specific physical and network conditions to run as intended. The architecture was developed under the following operational assumptions:

- **Track and surface:** The physical track must consist of a dark, continuous line placed on a distinctly lighter surface.
- **Lighting consistency:** Even lighting is required. Because the image processing utilises a threshold cut-off, sudden changes in illumination can lead to track loss.
- **Camera calibration:** The camera must be physically mounted and angled such that parts of the car are kept out of frame.

- **Car position:** The car must be aligned with the track's trajectory for it to recognise it as a path.

The applications can be run from the root directory of the project by running the command below on the client machine:

```
python3 picar_core/linefollower_app/linefollower.py
```

Upon execution, the processed camera feed pops up in a separate window (as seen in Figure 2.3). The frames have been modified to highlight the detected **path** contour in **yellow**, the **lines** fitted to the path in **green** and the planned **trajectory** of the car in **blue**.

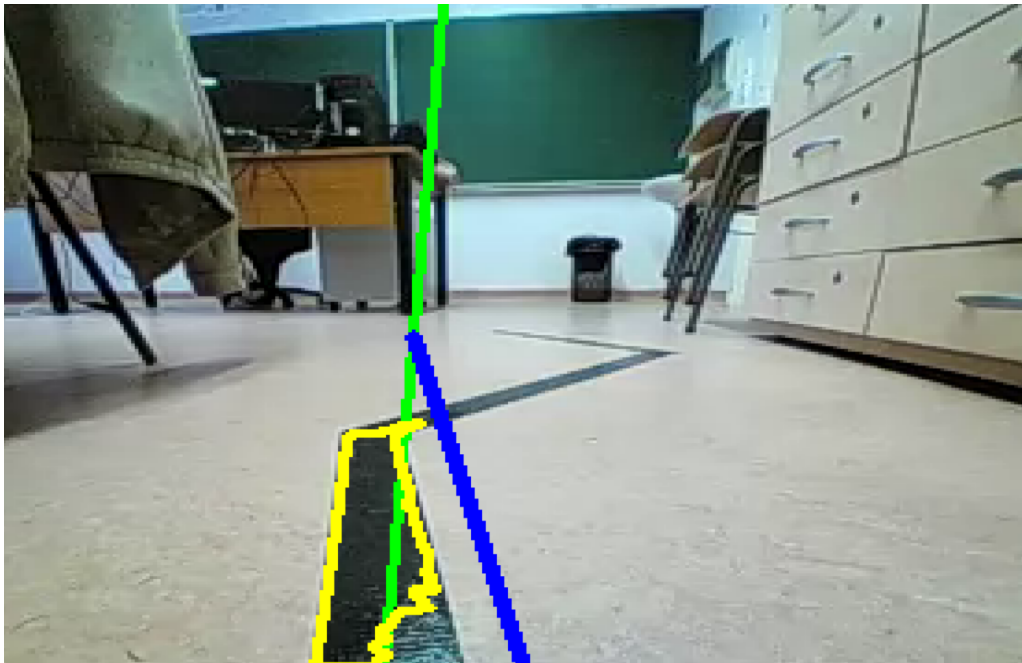


Figure 2.3: The visualiser feed of the line-follower application

The car starts following the path at a slow pace for safety. If the track is a closed loop (continuous), the car only stops once the button **Q** is pressed. If the track is point-to-point (finite), the car stops by itself once it reaches the end of the line.

Calibrating the algorithm for different conditions is possible via the `line_config.json` file.

Chapter 3

Developer documentation

3.1 Project structure

The working directory is organised as follows:

```
picar/
├── picar_core/ ... Core directory
│   ├── control_app/ ... Manual control interface
│   │   ├── client.html ... Frontend UI
│   │   ├── client.js ... Frontend control logic
│   │   └── config.js ... Frontend configuration
│   ├── linefollower_app/ ... Autonomous navigation module
│   │   ├── linefollower.py ... Line-follower script
│   │   └── line_config.py ... Line-follower configuration
│   ├── pc/ ... Relay server
│   │   ├── pc_server.py ... WebSocket relay
│   │   └── pc_config.json ... Relay server configuration
│   └── pi/ ... Raspberry Pi module
│       ├── hardware.py ... PWM and I2C motor/servo drivers
│       ├── pi_server.py ... Car main module
│       ├── pi_config.json ... Edge node configuration
│       ├── sensor.py ... Sensor interface
│       └── webrtc.py ... WebRTC media handling
├── tests/ ... Unit tests
└── pyproject.toml ... Project metadata
```

3.1.1 Configuration

The project utilises a Python packaging workflow centred around a `pyproject.toml` file. This configuration manages project metadata, defines the build environment, and specifies the necessary dependencies for the distributed system nodes.

Build specifications

The system is configured to use the `setuptools` build backend. The following command installs the project and its dependencies.

```
pip install .
```

Dependency management

The required libraries are specified within the `pyproject.toml` configuration file and are automatically resolved and installed during the project setup.

Command-line entry points

The project defines distinct command-line entry points to initialise the various system components. Because the core programs utilise asynchronous execution, these separate entry points were established to correctly manage the initialisation and execution of independent event loops for each node:

- **pi-server**: Launches the edge server, responsible for hardware control and camera streaming.
- **pc-server**: Launches the relay server, which facilitates signaling and data routing between the car and its clients.

3.2 Network architecture

The network architecture coordinates the interaction between the relay server, edge node, and client applications. By separating control signaling from media transmission, the system ensures low-latency execution and a stable video uplink.

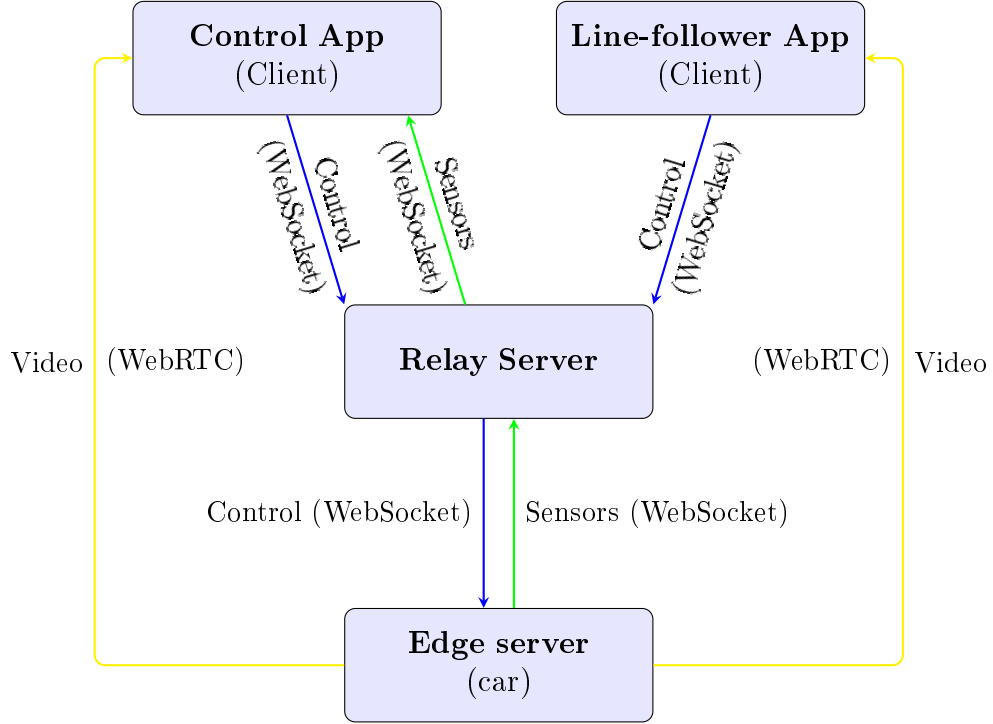


Figure 3.1: Communication flow between the client applications, the relay server, and the vehicle

The overarching communication flow between all active nodes in the system is illustrated by Figure 3.1. The control commands (blue arrows) sent by the clients are relayed by the central relay server to the edge server. Sensor data (green arrows) measured by the edge server is transported in the opposite direction through the relay server to the control application.

The video feed (yellow arrows), however, bypasses the relay server and gets transmitted straight to the clients. This peer-to-peer connection minimises latency and allows for high-speed video streaming.

While this high-level view demonstrates how the client applications, the relay server, and the vehicle interconnect, understanding the network requires deconstructing it into its individual components. At its core, the project manages three distinct data pipelines:

1. **Media pipeline:** WebRTC camera streaming, detailed in Section 3.2.1.
2. **Control pipeline:** The path for control commands, detailed in Section 3.2.2.
3. **Telemetry pipeline:** The publish-subscribe loop for sensor data, detailed in Section 3.2.3.

3.2.1 Media pipeline

The media pipeline provides the visual feedback necessary for vehicle operation. This section describes the signaling handshake and streaming protocols used to deliver the video feed to both clients.

Network initialisation

Communication between the relay server and the system nodes is established via the WebSocket protocol. Upon connection, participants send a `json` packet (seen in Code 3.1), identifying themselves as `client` or `car`, granting a transparent workflow.

```
{  
  "role": "client"  
}
```

Code 3.1: Identification packet defining the participant role.

The architecture establishes peer-to-peer media bridge between the vehicle and the client, using WebRTC. The central relay server coordinates the initial handshake protocol, after which the edge server streams the video feed directly to the client app, minimising latency.

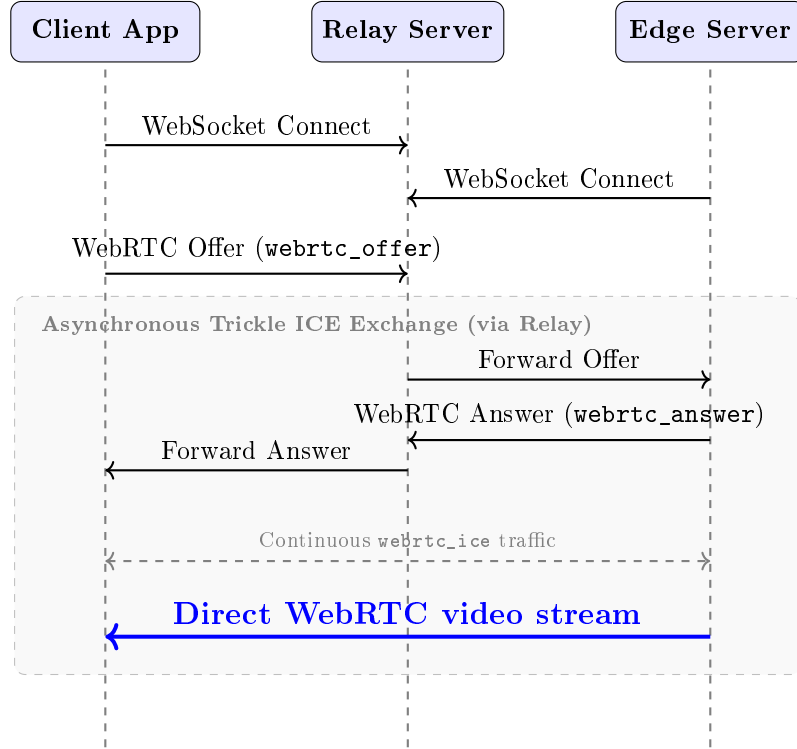


Figure 3.2: Sequence diagram illustrating WebRTC signaling handshake

As illustrated in Figure 3.2, the network is established with the WebSocket protocol. The client and the edge server send connection packages (`websockets.connect`) to the relay server.

Subsequently, the client initiates the handshake protocol by sending a WebRTC Offer through the relay server to the edge server. Concurrently, the trickle ICE protocol starts: the client and the edge server start exchanging ICE candidates with the other (example in Code 3.2), through the relay server.

The edge server sends back a WebRTC Answer through the relay server to the client, accepting the connection. Succeeding the transmission of the last ICE candidate, the edge server starts streaming the video stream directly to the client application.

```
1 {  
2   "type": "webrtc_ice",  
3   "candidate": {  
4     "candidate": "candidate:4234997325 1 udp 2043278322  
5       192.168.1.50 50000 typ host",  
6     "sdpMid": "video",  
7     "sdpMLineIndex": 0  
8   }  
}
```

Code 3.2: Example of an ICE candidate sent through the relay server

The edge server has transitioned from a signaling role to active media streaming. The camera mounted on the vehicle begins capturing the environment. This process prioritises high-speed transmission and consistent framerates over high resolution.

The termination point of this media pipeline changes depending on which client application is connected:

Manual Control Application:

When operated manually, the pipeline terminates in the browser. The video stream is directly decoded and displayed on the user interface, providing visual feedback, which is essential for navigation from a distance.

The specific software implementation and configuration of the Raspberry Pi's WebRTC media server are detailed further in Section 3.3.1.

Line-follower Application:

When operated autonomously, the video feed acts as the vehicle's primary sensor rather than a simple display. The pipeline routes the incoming frames to the line-follower application. The software calculates the required steering vectors. At the same time, it renders the processed frames in a dedicated window.

A comprehensive breakdown of the algorithm executed on these frames is provided in Chapter 3.3.4.

3.2.2 Control pipeline

The control pipeline is responsible for relaying control packets which direct vehicle movement. It is a one-way data stream that originates from the clients and terminates at the PCA9685 PWM controller.

This sequence is illustrated in Figure 3.3, which maps the transformation of high-level control logic into low-level hardware signals across the network architecture.

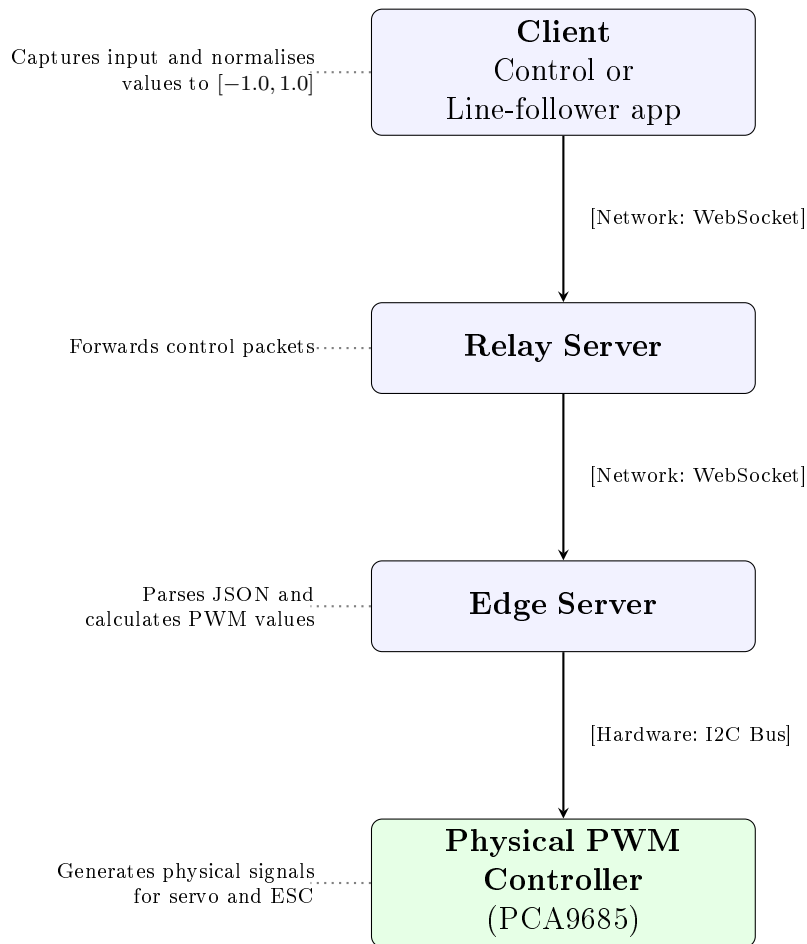


Figure 3.3: Data flow of the Control Pipeline

The pipeline consists of three primary software stages before final physical actuation.

1. **Input processing:** The client application (either the manual control or the line-follower application) calculates the desired control values. It normalises both steering and throttle into floating-point vectors ranging from -1.0 to 1.0 and formats them into a JSON string (example shown in Code 3.3).
2. **Control relay:** The central relay server receives the packet over the network. If an active `car_connection` exists, the server immediately forwards the unmodified string for minimising latency.
3. **Edge execution:** The Raspberry Pi extracts the JSON, transforms and transmits these values over the physical I2C bus to the PCA9685 chip. Implementation is detailed further in 3.3.1.

```
1 {  
2   "type": "control",  
3   "steer": 0.45,  
4   "throttle": -0.15  
5 }
```

Code 3.3: Format of the JSON control packet.

3.2.3 Telemetry pipeline

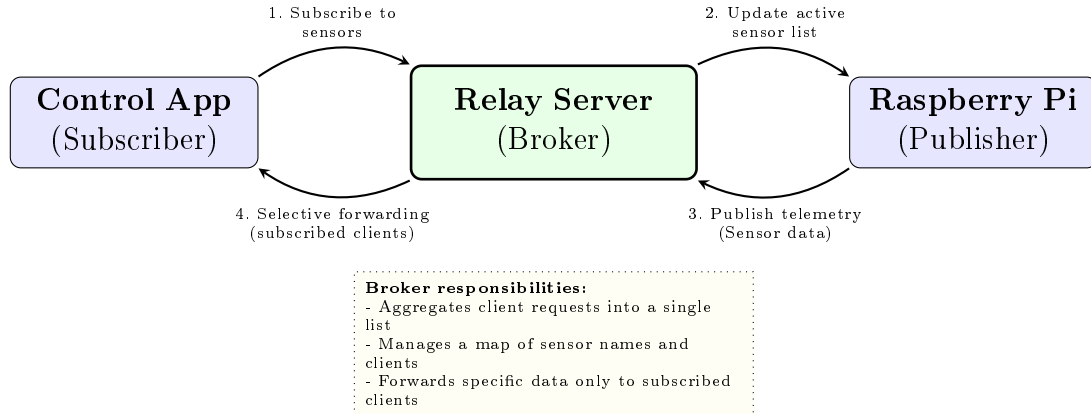


Figure 3.4: Publish-subscribe architecture for telemetry data

The telemetry pipeline operates on a custom publish-subscribe architecture, as illustrated in Figure 3.4. This design avoids transmitting superfluous sensor data to clients by ensuring they only receive the specific data streams they request. Similar architectures are frequently utilised in IoT sensor networks to optimise bandwidth.

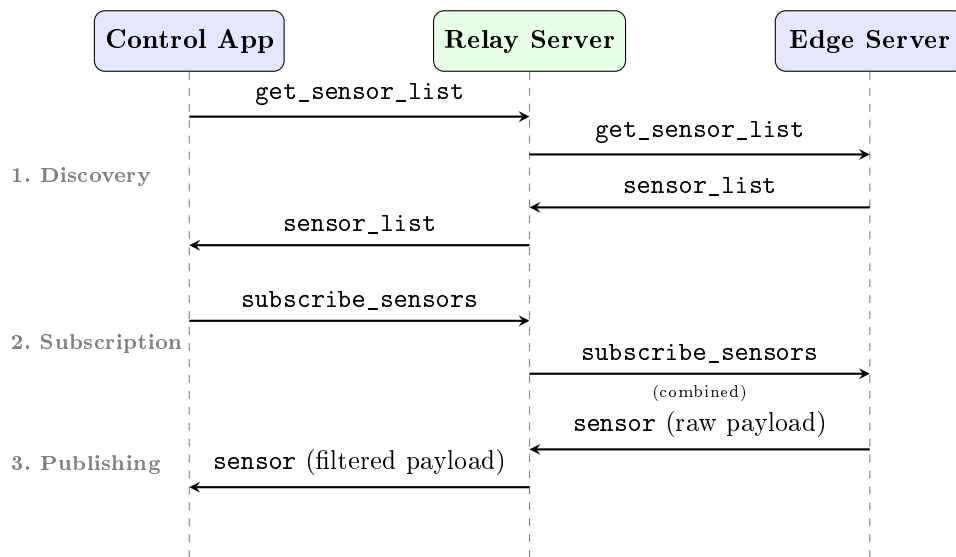


Figure 3.5: Sequence diagram detailing message flow of telemetry discovery and subscription

The process begins with a dynamic discovery phase, as seen in Figure 3.5. The client application requests a list of available sensors from the edge server (`json` format shown in Code 3.4).

```
1 {  
2   "type": "sensor_list",  
3   "sensors": ["cpu_core"]  
4 }
```

Code 3.4: Sensor discovery packet sent by the edge server.

Once the available actuators are identified, the control app subscribes to one or more sensors (topics). It sends this subscription request in a `json` format to the relay server, which acts as the broker in this model.

The relay server manages all subscriptions: it stores which clients are subscribed to which topics, and it dynamically updates the edge server regarding which sensors should currently be actively collecting data.

The edge server gathers data periodically, sends them to the relay server (publish), which distributes this data to subscribed clients. An example of a telemetry packet is shown in Code 3.5.

```
1 {  
2   "type": "sensor",  
3   "data": {  
4     "cpu_core": {  
5       "usage_percent": 45.2  
6     }  
7   }  
8 }
```

Code 3.5: Telemetry data packet containing raw sensor readings.

The specific software implementation of how sensor data is processed is detailed further in Section 3.3.1 (reading telemetry data), and in Section 3.3.3 (displaying telemetry data).

3.3 Implementation

The project utilises a file-based configuration system, in which data is dynamically loaded from `pi_config.json` on the Raspberry Pi, `pc_config.json` on the relay server and `config.js` within the control application. This allows for effortless testing with different parameters, both in regards to network and physical movement configurations.

3.3.1 Edge server

The edge server, running on the Raspberry Pi, serves as a bridge between the digital network and the physical world. Its primary responsibility is translating discrete control commands into hardware-level signals the car can utilise. It also processes the camera feed and reads telemetry from connected sensors.

Execution core (`pi_server.py`)

The hardware, sensor, and media modules are coordinated by `pi_server.py`, which acts as the execution core for the edge environment. Because the edge server must simultaneously stream video, monitor hardware safety, listen for driving commands, and handle telemetry, the server relies heavily on Python's `asyncio` library for concurrent execution.

The `pi_server.py` module initiates multiple asynchronous tasks alongside the main WebSocket listening loop:

- **Main loop:** Operating as the primary control dispatcher, the main loop receives incoming `json` packets and routes them based on their `type` key (example shown in Code 3.6). While WebRTC signaling and telemetry requests are offloaded to background tasks to prevent blocking, `control` commands are executed synchronously. This guarantees the lowest possible latency between network reception and hardware actuation. Prior to commanding the PWM driver, the loop updates the timestamp used by the `failsafe_task` and applies a deadzone filter to the input vectors to eliminate noise.

```
1 {  
2   "type": "control",  
3   "steer": 0.25,  
4   "throttle": 0.85  
5 }
```

Code 3.6: Example of an incoming JSON payload delegated by the main loop.

- **Sensor task:** The asynchronous `sensor_task` is responsible for planning the gathering of telemetry. To optimise system resources and network bandwidth, this task employs a selective polling strategy. It evaluates an active subscription map (`active_subs`) every second, querying the sensor only if a connected client has subscribed to that specific sensor. The gathered metrics are then serialised into JSON and transmitted to the relay server.
- **Failsafe task:** The asynchronous `failsafe_task` continuously monitors the timestamp of the last received control packet. If no command is received within the configured window, the system issues a neutral signal to both the steering and throttle channels, safely stopping the vehicle.

Once a valid control packet is parsed and cleared by the main loop's deadzone filters, the normalised vectors are passed down to the hardware driver for physical actuation, which is detailed in the following section.

Hardware interfacing (hardware.py)

The hardware interaction is contained in the `hardware.py` module. The architecture defines an abstract `OutputDriver` class, which is implemented by the `ServoHatDriver` class, as shown in the UML class diagram of Figure 3.6. This driver manages communication over the I2C bus utilising the `smbus2` library to interface with a PCA9685 PWM controller.

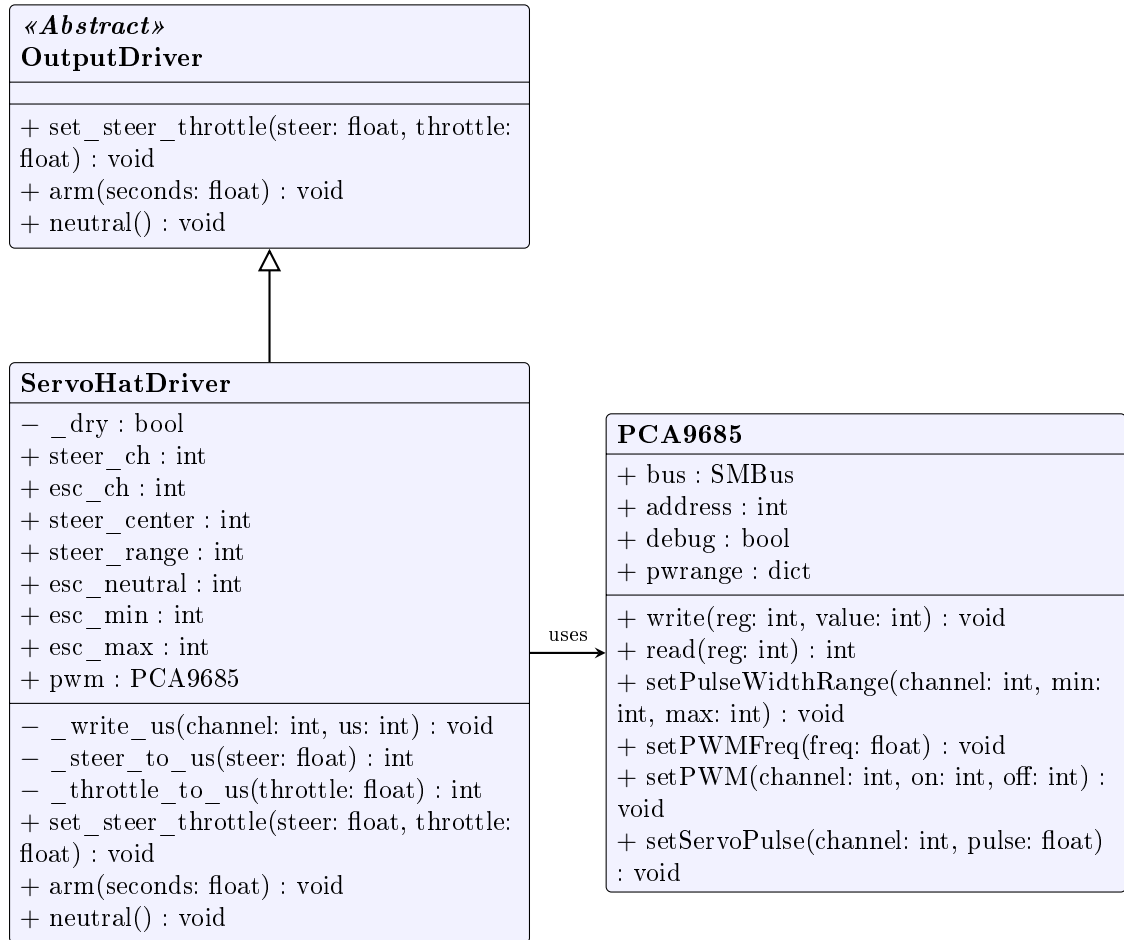


Figure 3.6: UML class diagram illustrating `hardware.py`

Beyond the basic inheritance structure, the `ServoHatDriver` maintains several state variables and private helper methods designed for operational safety. The `_dry` flag allows the system to initiate in a simulation mode if the physical I2C bus is unreachable, preventing software crashes during testing.

Moreover, the class contains the logic required to transform normalised steering and throttle inputs into microsecond-based pulse widths through its private `_steer_to_us` and `_throttle_to_us` methods. These conversions rely on hardware-specific parameters such as center points and range limits, which ensure that the signals remain within the physical tolerances of the servo and ESC.

PWM Signal Translation

Control commands sent by the client applications arrive at the edge server as normalised floating-point vectors (ranging from -1.0 to 1.0). The `ServoHatDriver` maps these into the absolute microsecond pulse widths required by the physical PWM controller.

The parameters for the neutral baseline, the forward and the reverse limits (maximum and minimum pwm) for the ESC can be set in `pi_config.json`, as well as the range and center point of the servo. These values have been calibrated for stable control dynamics, but the variables can be adjusted for different purposes.

Furthermore, a software-level deadzone for steering and throttle is enforced within `pi_server.py` to discard insubstantial control packets to minimise unnecessary noise.

ESC arming

Upon initialisation, the `ServoHatDriver` executes an arming sequence that holds the throttle at the neutral position for two seconds, satisfying the ESC's safety interlock requirements before it accepts active driving commands. This is a crucial safety measure that prevents the vehicle from performing dangerous, unintended movements by ensuring the ESC only activates after a stable neutral signal is confirmed.

Sensor Integration (`sensor.py`)

Beyond executing physical commands, the edge server is responsible for gathering sensor data. To support future hardware additions, the project utilises an extensible architecture for sensor integration, illustrated in Figure 3.7.

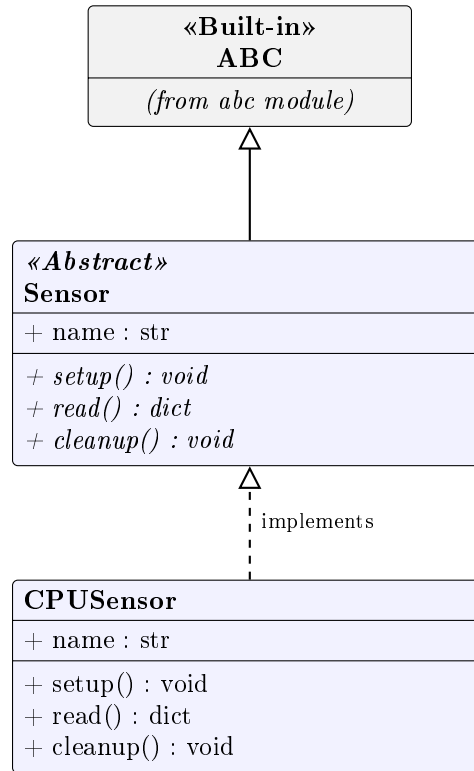


Figure 3.7: UML class diagram illustrating `sensor.py`

The `abc` (Abstract Base Class) library was used for the reason that Python works differently than most other object-oriented programming languages. Since Python natively relies on "duck typing," it checks interface compliance at runtime, when the methods are called. By explicitly inheriting from `ABC`, the architecture enforces strict constraints. This measure guarantees that any future sensor implementation will trigger an immediate instantiation error if it fails to provide the required methods, preventing crashes during operation.

Lifecycle management

While the `read()` method handles data acquisition, the abstract interface also enforces `setup()` and `cleanup()` methods to manage hardware state. This lifecycle management is critical for working with physical actuators.

The `setup()` method ensures that required protocols (such as I2C buses) are properly initialised before the asynchronous polling loop begins. Conversely, the `cleanup()` method guarantees that pins and communication buses are safely released during a system shutdown or crash, thus preventing hardware lock-ups.

Sensor emulation

To showcase the infrastructure, a CPU utilisation sensor (`CPU_Sensor`) is emulated. This module uses the `psutil` library to retrieve CPU usage percentages directly from the Raspberry Pi's operating system, then forwards it to the relay server (the operation of this network is discussed in Section 3.2.3).

Asynchronous Polling and Transmission

To guarantee that data acquisition does not block the primary WebSocket control thread, the edge server handles sensor reading asynchronously. A dedicated background task periodically polls the active sensors at a configured frequency.

Once the raw data is read from the hardware pins, it is normalised, packaged into JSON payloads, and pushed upstream to the relay server over the established WebSocket connection. This continuous stream of telemetry allows the client applications - particularly the line-follower application - to make navigational decisions based on live physical data.

Camera processing (webrtc.py)

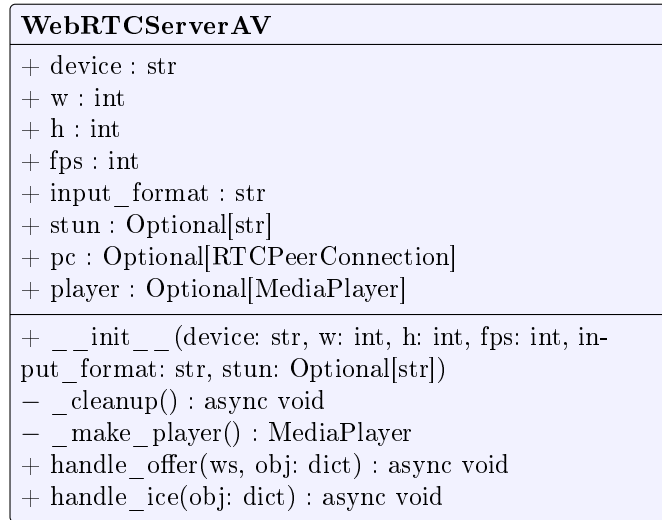


Figure 3.8: UML class diagram illustrating the **WebRTCServerAV** class

The **WebRTCServerAV** class, illustrated in Figure 3.8, handles both getting the feed from the physical camera and establishing the video transmission with the help of **aiortc**, a Python implementation of **WebRTC**.

The media capture and signaling implementation in this project has been designed based on the official **aiortc** documentation and examples [3].

Hardware media capture

To interface with the physical camera, the class implements the **_make_player()** method, which utilises the **MediaPlayer** module from **aiortc**. The capture parameters, including resolution, framerate, the **yuyv422** input format and the camera address, are set dynamically during instantiation, allowing them to be easily managed via the **pi_config.json** file without modifying the source code.

Video stream transmission

The class realises two asynchronous methods to establish connection with the clients:

- **handle_offer()**: Triggered when the edge server receives a **WebRTC** offer from a client. It configures the **RTCPeerConnection**, attaches the camera's video track to the peer connection, and generates the **WebRTC** answer to complete the initial handshake. Once established, the video gets transmitted continuously until the session is terminated.

- `handle_ice()`: Processes ICE candidates received from the client. These candidates contain potential routing addresses discovered by the client. Adding them to the established peer connection optimises the media routing path and reduces initial connection latency.

NAT traversal

To ensure the video stream can be established when the edge node and client are situated behind different local routers or firewalls, the class supports the use of a STUN server. The STUN address, defined in the configuration file, is passed to the `WebRTCServerAV` instance and applied to the `RTCConfiguration` during the connection handshake. This allows the edge server to discover its public-facing IP address and generate reachable ICE candidates, facilitating peer-to-peer media transmission.

Non-blocking negotiation

The WebRTC handshake process can be computationally expensive. To prevent these operations from stalling the primary event loop - which would disrupt the low-latency processing of incoming control commands - the edge node dispatches the `handle_offer()` routine as an independent background task. This concurrent execution guarantees that the vehicle remains fully responsive to driving inputs even during the initialisation or recovery of the video stream.

Resource cleanup

To ensure stability against sudden network drops or client disconnections, the `WebRTCServerAV` class implements a `_cleanup()` routine. Executed automatically before processing any new connection offer, this method safely stops all active media tracks, closes the `MediaPlayer`, and terminates any lingering `RTCPeerConnection` states, ensuring a safe connection between the edge server and the clients.

3.3.2 Relay server

The relay server `pc_server.py`, hosted on a central computer, acts as the communication backbone of the system. Because WebRTC requires an initial signaling phase to negotiate network paths and media formats before establishing a peer-to-peer connection, a central WebSocket gateway is necessary to manage this handshake. Beyond WebRTC signaling, the relay server is responsible for transmitting control commands and managing telemetry subscriptions. It uses Python's `websockets` library [4] to communicate with the edge server and the clients. The server handles concurrent connections through a unified event loop.

Role-based identification

When a new WebSocket connection is established, the server waits for an initial `json` payload containing a `role` key. Based on this identifier, the server designates the connection as either the singular `"car"` or adds it to a dynamic set of `"client"` connections.

This strict separation ensures the server knows exactly where upstream and downstream data should be routed. The reason why `json` is the chosen format - while it can evidently be considered wasteful compared to other methods for classification - is because it is well observable, allowing for future demonstrations.

Bidirectional message transfer

The server acts as a transparent relay center for different types of `json` messages:

- **Upstream (Client to Car):** Driving commands (`steer`, `throttle`) and client-generated WebRTC signals (Offers, ICE candidates) are forwarded directly to the edge server.
- **Downstream (Car to Client):** The car's WebRTC signaling (Answers, ICE candidates) is broadcast to all the connected clients, while telemetry data is only forwarded to subscribed clients.

Selective telemetry forwarding

The server implements a publish-subscribe architecture to create a scalable environment that is able to handle several clients and sensors. The exact flow of the telemetry data is discussed in 3.2.3, this section focuses on its implementation at the relay server.

Subscription management

When clients send `subscribe_sensors` messages detailing which sensors' data is required, the relay server stores these requests in a Python `defaultdict(set)`, which efficiently maps each sensor's unique string identifier (e.g. `"cpu_core"`) to a set of active client `websocket` objects.

The use of `defaultdict(set)` allows for clean management of this map, as unlike a normal Python `dictionary`, it does not raise an error when trying to access a nonexistent key. Instead, it creates a new set for the missing key. This is highly useful for one-to-many mappings, like the subscriptions map in this project, as it eliminates the need to manually check whether a key exists or not.

To maintain an accurate state, if a client disconnects, the server immediately iterates through this registry to remove obsolete subscriptions and empty keys.

Selective payload forwarding

When the edge server transmits a `sensor` payload, the relay server unpacks the data and performs selective forwarding. Instead of simply broadcasting the entire dictionary, it queries the subscription registry for each specific sensor identifier. It sends the data to all subscribed clients using the `websockets.broadcast()` method. Unlike its name suggests, this method does not broadcast the payload to all clients, the set of receivers can be passed as its first parameter.

Edge synchronisation

To minimise the toll on edge computing resources, the relay server dynamically synchronises the active subscription list with the edge environment. Every time the global subscription registry is modified (e.g. a client joins, alters preferences, or disconnects), the server executes the asynchronous `update_car_subscriptions()` task. This function collects the currently active dictionary keys and transmits an updated `subscribe_sensors` list to the Raspberry Pi.

3.3.3 Manual control application

The control application serves as the primary user interface for the system, accessible via a standard web browser. It is built with HTML (`client.html`), CSS (`style.css`), and JavaScript `client.js`, responsible for presenting the live video feed, capturing and forwarding control inputs, and displaying telemetry data.

Connection and media rendering

Upon loading, the application establishes a WebSocket connection to the relay server and registers its role as a "client". Immediately following this initial handshake, the application instantiates an `RTCPeerConnection` object and generates a WebRTC offer. This offer, along with subsequent ICE candidates, is routed through the relay server to the Raspberry Pi. Once the peer-to-peer negotiation is successful, the incoming video stream is directly bound to an HTML5 `<video>` element, providing the operator with a low-latency view from the car's perspective. This media transfer is further detailed in Section 3.2.1.

The client interface provides also internal diagnostic feedback. The `tick()` function tracks the total number of WebSocket messages dispatched per second, updating a live "Rate (Hz)" indicator on the dashboard. This metric enables verification of the client's execution speed and the transmission rate of the control signals.

Input processing loop

The most straight-forward implementation of keyboard controls would send discrete commands only when a key down or key up event fires. This would result in shaky, binary vehicle movement.

To avoid this, the control application implements a continuous animation loop utilising the built-in `requestAnimationFrame()` method. This method requests that the browser call a specific function (in this case, the `tick()` function) synchronised with the display's refresh rate.

During each tick, the application evaluates the current state of a keyboard `Set` (tracking standard WASD or arrow keys) and applies smoothing:

- **Steps:** If a directional key is actively pressed, the system increments or decrements the steering and throttle values by configurable step sizes (`sStep` and `tStep`).

- **Auto-centering:** If no keys are pressed, the system automatically applies an inverse step to return the steering and throttle variables back to neutral.
- **Clamping:** The variables are clamped to a normalised range of $[-1.0, 1.0]$.

Step sizes can be set in the input fields of `client.html`. The client automatically validates the input, if it is not a number or it is not in range $[0.01, 0.5]$, it is set back to 0.06.

Pressing a control button dynamically updates the indicator bars before the control packet is sent into a `json` payload and transmitted over the WebSocket to the relay server.

Dynamic telemetry interface

The application seamlessly integrates with the publish-subscribe architecture detailed in Section 3.2.3. Upon connecting, it sends a `get_sensor_list` message to discover available telemetry streams. The response is used to dynamically generate a series of UI checkboxes, allowing the user to select which data points they wish to monitor.

When the subscribed `sensor` payloads arrive, the frontend extracts the values, applies formatting rules defined in `config.js` (such as decimal precision and suffixes (e. g. "%")), and updates the display asynchronously without disrupting the primary control loop.

3.3.4 Line-follower application

Besides manual controlling, the system is also capable of basic autonomous navigation. This is achieved through a dedicated Python application (`linefollower.py`) that intercepts the incoming WebRTC video stream and applies an algorithm to track a physical line on the ground. See Section 2.4.2 for details on environment requirements.

Figure 3.9 illustrates the step-by-step transformation of raw video data into calculated steering and throttle commands.

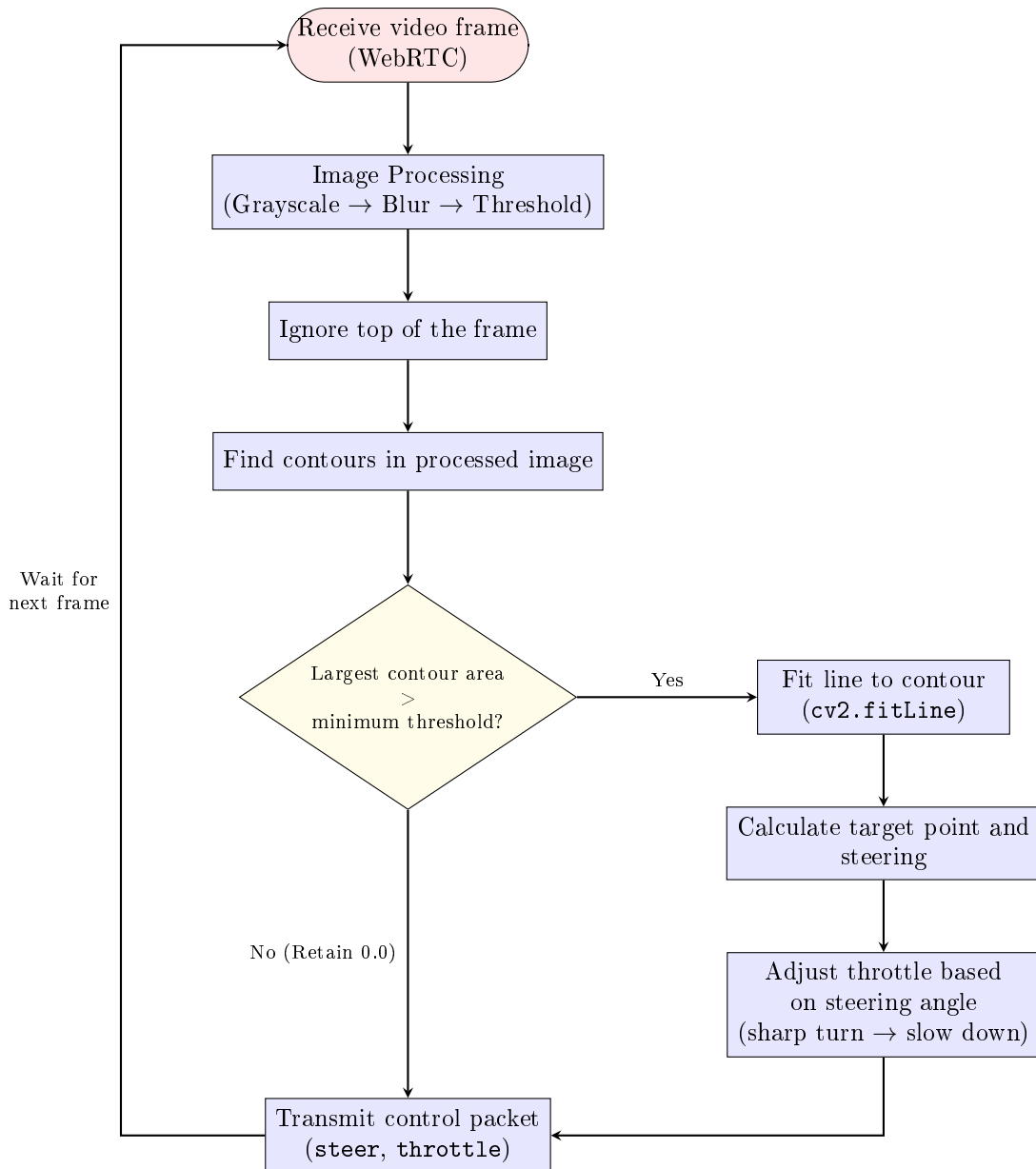


Figure 3.9: Flowchart of the line-follower algorithm

Pipeline explanation

The autonomous control loop is implemented inside the asynchronous `process_video()` function, utilising the OpenCV library to process the image. The execution of the application follows the following steps:

1. **Converting to grayscale:** The raw video image is converted to grayscale (`cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`). This significantly reduces computational load.
2. **Blurring:** A Gaussian blur is applied (`cv2.GaussianBlur()`). This reduces noise, so the program does not mistake parts of the background as our path.
3. **Applying threshold:** An inverse binary threshold (`cv2.THRESH_BINARY_INV`) is applied. This inverts dark pixels (values below the threshold) into pure white, and lighter pixels to pure black (0). If the environment was optimal, we now have a white path on a fully black background.
4. **Masking:** To prevent the algorithm from being confused by distant objects or background noise at the horizon, the top of the frame is masked by forcing the pixel values to zero. This ensures the car only reacts to the path immediately in front of it.
5. **Contour extraction:** The algorithm utilises `cv2.findContours()` to map the edges of the white shapes in the thresholded image.
6. **Selecting largest contour:** The largest contour by area is identified. If the environment was optimal, this contour marks our path. As a fail-safe measure, if the largest contour is smaller than the set threshold, it is discarded, the steering and throttle values default to 0.0, stopping the car.
7. **Fitting a line:** If a valid line contour is found, the script first uses `cv2.fitLine()` to mathematically fit a straight line directly through the contour representing the physical path.
8. **Calculate target point:** By extending the line, the script calculates exactly where the path intersects a horizontal line across the camera frame (determined by the look-ahead percentage), establishing a target point to steer towards.

9. **Calculate steering:** To calculate the required steering angle, we first define the normalised error $e(t)$ between this target point and the physical center axis of the car. This value is then further transformed by a proportional–integral–derivative (PID) control mechanism [5]. The control signal $u(t)$ is computed using the following formula:

$$u(t) = K_p e(t) + K_i \sum e(t) + K_d \Delta e(t)$$

Where K_p , K_i , and K_d represent the proportional, integral, and derivative tuning gains.

10. **Controlling speed:** Instead of driving at a constant speed, the pipeline adjusts the throttle based on how sharp the turn is. If the steering value indicates a sharp turn (greater than 0.6), the throttle duty cycle is reduced, whereas straighter paths allow an 80% duty cycle.

Once the parameters are calculated and clamped to their valid $[-1.0, 1.0]$ ranges, the application transmits a `json` control package back through the WebSocket relay server to execute the physical movement.

Fine-tuning

All the parameters of the algorithm can be set in the `line_config.json` file. The following parameters are particularly useful for configuring the application for different conditions:

- **look_ahead_percent:** The height of the calculated target point. Setting this to 1 would mean the target point is set to the bottom of the image, while 0 would be the top of the image.
- **crop_top_percent:** This sets the percentage of the image height from the top that should be disregarded (masked) and excluded from calculations.
- **threshold_min:** The cut-off value of the binary threshold. Values below this are changed into pure white, values over it into black. Useful to adjust for different lighting conditions.

- `duty_cycle_sharp` and `duty_cycle_straight`: Configurable integers between 1 and 10. These parameters specify the exact number of active throttle frames per 10-frame window for sharp turns versus straight lines.
- `kp`: The proportional gain. Represents how sharply the car steers the further it is horizontally from the target point.
- `ki`: The integral gain. Represents how sharply the car steers in relation to the increasing duration of horizontal misalignment to the target point.
- `kd`: The derivative gain. Represents how quickly the car is getting to the target point. If it is higher, the car slows down more as it gets closer to the target point, to avoid overshooting.

Engineering decisions

PID mechanism:

The steering control was originally implemented as a simple proportional (P) controller, where the steering angle was strictly a multiple of the current error. The architecture was upgraded to a full PID controller since it is far more tunable, resulting in accurate line following.

Car speed:

As detailed in Step 10, speed reduction during turns is achieved by throttling the command duty cycle rather than simply scaling down the motor's power output variable. This decision addresses the physical limitations of the used motor, which lacks the ability to function at low voltage levels. By transmitting intermittent pulses of higher power rather than a continuous low-power signal, the vehicle is able to traverse the path at a slow, safe speed.

Latency:

Because this autonomous navigation relies on a distributed architecture, the control loop is subject to network latency. The delay between the physical camera capturing a frame, transmitting, processing and transmitting it back to the edge node causes latency. The dynamic throttle management detailed in Step 10 helps mitigate this latency by recognising sharp turns and forcing the vehicle to slow down, giving the pipeline more time to process frames and calculate corrections before the vehicle slips off the track.

Path discovery:

The final path-detection logic is the result of an iterative design process. Initially, the steering vector was calculated using only the center of the track's contour. This approach struggled with sharp corners, as the center point simply averages the path, causing the vehicle to often turn too late. A second iteration bypassed contours entirely, applying the line-fitting algorithm (`cv2.fitLine`) directly to the preprocessed image. However, this proved overly sensitive to environmental noise, a single dark object in the background would skew the drawn line. The final, combined solution resolves both issues: it uses contour detection as a noise filter to isolate the track, and then applies line fitting exclusively to that bounded shape. This results in a highly stable, predictive steering vector that is resilient to visual interference.

3.4 Testing

This section details the tests that were performed in order to ensure the code works properly, even considering edge cases. Two types of tests were executed during the development of this software: unit tests to verify code in the background works as expected, and manual tests to confirm the intended outcome.

3.4.1 Unit tests

Writing unit tests is an effective method to ensure the reliability and mathematical correctness of the core modules. In this project, an automated test suite of 13 tests was implemented using the `unittest` framework, utilising mocks to isolate software logic from physical hardware.

These tests are located in the `test_hardware.py` and `test_server.py` files within the `tests` subfolder.

Executing the test suite

The automated test suite can be executed from the project root using the following command:

```
python -m unittest discover -s tests
```

Hardware driver testing

The hardware tests primarily validate the `ServoHatDriver` class to ensure that software commands are correctly translated into the physical signals required by the ESC and the servo. Since automated test pipelines lack the physical I2C bus and the PCA9685 controller chip, the tests rely on the `unittest.mock` library to resolve this deficiency.

During test initialisation, the `smbus2` module is replaced with a `MagicMock` object, allowing the program to be tested in complete isolation from the hardware. The test suite systematically verifies the internal mathematical conversion functions and the safe running of the program.

- **Steering PWM conversion:** The `_steer_to_us` method is tested to ensure normalised directional inputs correctly map to the configured microsecond limits. The tests assert that a neutral 0.0 input strictly produces the 1500 microseconds center pulse. Furthermore, boundary tests confirm that the configured mechanical range is correctly applied when giving full left or full right commands.
- **Throttle PWM conversion:** The `_throttle_to_us` method is tested to evaluate accurate motor operation. The tests verify that a 0.0 input outputs the exact neutral 1500 microsecond signal necessary to keep the vehicle stationary. Maximum acceleration and maximum reverse/braking are confirmed to be correctly mapped to the boundary limits.
- **Dry run:** By mocking the `PCA9685` object and forcing it to raise an "I2C Bus not found" exception, the test case verifies that the application falls into a `dry_run` simulation mode.

Routing logic testing

The relay server tests utilise `AsyncMock` objects to validate the publish-subscribe architecture:

- **Message routing:** The tests verify that control commands are routed to the car and WebRTC signals are broadcasted to all clients.
- **Selective forwarding:** The tests confirm that sensor payloads are forwarded only to clients that are subscribed to the specific sensor and that the car is notified of subscription changes.
- **Clean-up:** The tests ensure that the server cleans up the subscription registry and `websocket` objects immediately upon client disconnection.

3.4.2 Manual tests

To ensure the system operates safely and effectively in practice, comprehensive manual testing was conducted throughout development. Because the project involves physical hardware, automated tests alone are insufficient to validate the functionality of the software.

The manual testing procedures focused on several areas:

- **UI responsiveness and command transmission:** The controlling of the car through the web-based control application was tested.
- **Video streaming:** The WebRTC video feed was subjectively evaluated to ensure latency remained low enough for safe remote driving.
- **Hardware tuning:** Configurations for the bounds of steering and throttle PWM signals were worked out to ensure safe mechanical operation and minimise hardware strain.
- **Failsafe:** Safety mechanisms were tested by deliberately severing the connection to ensure the movement of the vehicle completely stopped.
- **Line-following:** The line-following program was tested in multiple conditions to ensure the development of an efficient algorithm.

Manual testing of control app inputs

Table 3.1 summarises the testing scenarios of the manual control application. The keys were held for an extended period of time to reach the maximum boundaries and provide a fair comparison. The steering and throttle commands were intercepted at the edge server to confirm the accurate transmission of commands.

Key(s)	Throttle	Steering	Response
W (hold)	1.00	0.00	The car is moving forward at maximum speed
S (hold)	-1.00	0.00	The car is moving backward at maximum speed
A (hold)	0.00	-1.00	The steering turns to the full-left position
D (hold)	0.00	1.00	The steering turns to the full-right position
W + D	1.00	1.00	The car moves forward while turning to the right
W + A	1.00	-1.00	The car moves forward while turning to the left
S + D	-1.00	1.00	The car moves backward while turning to the right
S + A	-1.00	-1.00	The car moves backward while turning to the left
W + S	0.00	0.00	Opposing inputs cancel: car remains stationary
A + D	0.00	0.00	Opposing inputs cancel: steering remains centred

Table 3.1: Manual testing results of control logic

Chapter 4

Conclusion

This project achieved the objective of developing a software control platform for a remote-controlled car. In its current state, the system supports two distinct modes of operation.

It provides a web-based manual control interface. Keyboard input commands are captured through the client and get transmitted to the car, controlling it. This is paired with a direct peer-to-peer video streaming network, providing visual feedback.

The other client is a line-follower application that processes the live video feed to track the path, autonomously sending control commands to the car. This mode utilises a PID control loop to adjust steering and speed based on the calculated trajectory error.

Together, these features establish a functional baseline for both remote and autonomous vehicle operation.

4.1 Future plans

The system's modular design makes the platform extensible, ideal for future studies and educational demonstration. The sensor interface makes it possible to implement different sensors, providing several opportunities for new projects.

As the platform scales, one area worth exploring could be the addition of a client administration module to make the system more secure. Additionally, while the current architecture relies on a direct peer-to-peer WebRTC connection, a potential architectural shift could involve routing the camera feed directly through the central relay server. This centralised approach could allow the server to broadcast video to multiple observers simultaneously, without overloading the vehicle's onboard network bandwidth. However, the viability of this routing method would require testing to ensure that the resulting latency remains within acceptable bounds for remote navigation.

Acknowledgements

I would like to express my gratitude to my supervisor, Laki Sándor, for allowing me the opportunity to work on this project. I am particularly grateful for the academic freedom and creative independence he granted me throughout the development process, which allowed me to develop this platform according to my vision.

Bibliography

- [1] Modell.hu. *Phantom MT 1/10 RTR Monster Truck*. Accessed: 2026-04-23. URL: <https://shop.modell.hu/phantom-mt-1-10-rtr-monster-truck.html>.
- [2] RS Components. *Raspberry Pi 3 Model B+*. Accessed: 2026-04-23. URL: <https://hu.rs-online.com/web/p/raspberry-pi/1373331>.
- [3] Jeremy Lainé. *aiortc Documentation*. Accessed: 2026-04-17. 2024. URL: <https://aiortc.readthedocs.io/>.
- [4] Aymeric Augustin. *websockets Documentation*. 2024. URL: <https://websockets.readthedocs.io/> (visited on 04/23/2026).
- [5] Norman S. Nise. *Control Systems Engineering*. 7th. Hoboken, NJ: John Wiley & Sons, Inc., 2015. ISBN: 978-1-118-80082-9.

List of Figures

2.1	The client before connecting to the server	11
2.2	The client after connecting to the server	12
2.3	The visualiser feed of the line-follower application	14
3.1	Communication flow between the client applications, the relay server, and the vehicle	17
3.2	Sequence diagram illustrating WebRTC signaling handshake	19
3.3	Data flow of the Control Pipeline	21
3.4	Publish-subscribe architecture for telemetry data	23
3.5	Sequence diagram detailing message flow of telemetry discovery and subscription	23
3.6	UML class diagram illustrating <code>hardware.py</code>	27
3.7	UML class diagram illustrating <code>sensor.py</code>	29
3.8	UML class diagram illustrating the <code>WebRTCServerAV</code> class	31
3.9	Flowchart of the line-follower algorithm	37